A MINIMAL FIELD HANDBOOK

The Minimal Multimodal & Diffusion Handbook

Diffusion, flow matching, CLIP conditioning and vision-language models - the generative stack.

Mosafer

diffusers · CLIP · latent diffusion · VLMs

V1.0 - COMPACT EDITION

NOVEMBER 2026

Contents

1 Orientation **2**
- 1.1 The map . . . 2
- 1.2 The toolchain . . . 2
- 1.3 The unifying view . . . 2

2 Diffusion from Scratch **3**
- 2.1 The forward process (fixed, no learning) . . . 3
- 2.2 The training objective . . . 3
- 2.3 The reverse process (the generation loop) . . . 3

3 Samplers and Flow Matching **5**
- 3.1 DDIM: deterministic, few steps . . . 5
- 3.2 The sampler menu . . . 5
- 3.3 Flow matching: the modern formulation . . . 5

4 Conditioning: CLIP, Cross-Attention, Guidance **7**
- 4.1 CLIP: the shared space . . . 7
- 4.2 Cross-attention: where text enters the image . . . 7
- 4.3 Classifier-free guidance (CFG) . . . 7

5 Latent Diffusion and Fine-Tuning **9**
- 5.1 The architecture . . . 9
- 5.2 LoRA fine-tuning . . . 9

6 Vision-Language Models **11**
- 6.1 The architecture (LLaVA-style) . . . 11
- 6.2 Evaluation . . . 11

A Cheat Sheet **13**
- A.1 Diffusion . . . 13
- A.2 Latent diffusion . . . 13
- A.3 VLMs . . . 13
- A.4 Idioms worth memorizing . . . 13

CHAPTER 1

Orientation

Multimodal models connect perception to language along two arcs: *generation* (diffusion models that draw) and *understanding* (vision-language models that see). This handbook derives diffusion from scratch, builds the conditioning machinery both arcs share, and finishes with VLMs and their evaluation.

1.1 The map

Chapter	Content
2 diffusion	the forward/reverse process, the DDPM training objective
3 sampling	DDIM, schedulers, and flow matching — the modern formulation
4 conditioning	CLIP embeddings, cross-attention, classifier-free guidance
5 latent diffusion	the VAE + UNet architecture of Stable Diffusion, LoRA fine-tuning
6 vision-language models	contrastive training, projector LLMs, evaluation

1.2 The toolchain

```
pip install diffusers transformers accelerate safetensors
pip install clip open_clip_torch
```

`diffusers` is to image models what `transformers` is to text: one API (`from_pretrained`), pipelines for quick use, components underneath.

1.3 The unifying view

Both arcs learn *conditional distributions*: $p(\text{image} \mid \text{text})$ for generation, $p(\text{text} \mid \text{image})$ for understanding — and both train on paired data scraped at scale. The conditioning machinery (Ch. 4) is the hinge: CLIP's embedding space is where text and image meet, in diffusion guidance and in VLM projectors alike.

> **PITFALL**
> Diffusion and VLMs are trained at scales where the GPU-hours matter (Ch. 6 of the Distributed handbook applies). Fine-tuning, not pretraining, is this book's mode: LoRA on a pretrained Stable Diffusion or a projector on a pretrained VLM — the recipes here run on one 24 GB card.

CHAPTER 2

Diffusion from Scratch

Diffusion learns to generate by learning to *destroy*: corrupt images with noise step by step, then train a network to undo one step of the corruption. Generation is running the undo loop in reverse.

2.1 The forward process (fixed, no learning)

Add Gaussian noise in T steps, scheduled by $\beta_1, \ldots, \beta_T$:

$$q(x_t \mid x_{t-1}) = \mathcal{N}(x_t; \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I)$$

The closed form jumps to any timestep directly — the practical gift of the Gaussian chain:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} (1-\beta_s)$$

At $t = T$, $\bar{\alpha}_T \approx 0$: the image is pure noise.

2.2 The training objective

A network $\epsilon_\theta(x_t, t)$ predicts the noise that was added. The objective is plain MSE on that prediction:

$$\mathcal{L} = \mathbb{E}_{x_0, \epsilon, t} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t) \right\|^2$$

```
import torch, torch.nn as nn

def training_step(unet, x0, t, betas_bar_alpha):
    # t sampled uniformly; noise drawn fresh every step
    eps = torch.randn_like(x0)
    ab = betas_bar_alpha[t].view(-1, 1, 1, 1)
    x_t = ab.sqrt() * x0 + (1 - ab).sqrt() * eps
    pred = unet(x_t, t)
    return nn.functional.mse_loss(pred, eps)

# that is the entire DDPM loss - everything else is sampling
```

2.3 The reverse process (the generation loop)

Start from noise $x_T \sim \mathcal{N}(0, I)$; step backward with the predicted noise:

$$x_{t-1} = \frac{1}{\sqrt{1-\beta_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\, \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

```
@torch.no_grad()
def sample(unet, betas, abar, T=1000):
    x = torch.randn(1, 3, 32, 32)          # x_T
    for t in reversed(range(T)):
        ab, ab_prev = abar[t], abar[t-1] if t > 0 else 1.0
```

```
        eps = unet(x, torch.tensor([t]))
        x0 = (x - (1 - ab).sqrt() * eps) / ab.sqrt()   # implied x0
        x = ab_prev.sqrt() * x0 + (1 - ab_prev).sqrt() * eps
    return x
```

PITFALL

The subtleties that bite: (1) the `t`-embedding must be *conditioned in* (sinusoidal/learned encodings) — the same network handles every noise level; (2) the loss is over *random* timesteps per batch element — not sequential; (3) predicting noise vs. predicting x_0 vs. velocity (Ch. 3) are reparametrizations of one objective — frameworks differ, papers mix conventions; read which one a checkpoint uses.

CHAPTER 3

Samplers and Flow Matching

DDPM sampling needs 1000 steps; DDIM gets the same images in 20–50; flow matching reframes the whole thing as straight-line motion and is the modern default (Stable Diffusion 3, Flux). This chapter is the sampler zoo with the math that separates them.

3.1 DDIM: deterministic, few steps

DDIM drops the stochastic term and reparametrizes sampling as moving along the predicted x_0: jump from t to any earlier t' directly:

$$x_{t'} = \sqrt{\bar{\alpha}_{t'}}\ \underbrace{\hat{x}_0(x_t)}_{\text{predicted}} + \sqrt{1-\bar{\alpha}_{t'}}\,\epsilon_\theta(x_t, t)$$

Deterministic (same seed \(\to \) same image), which also makes *inversion* possible: encode a real image back to noise by running the update in reverse — the basis of image editing (img2img, prompt-to-prompt).

3.2 The sampler menu

Sampler	Character
DDPM	1000 steps, stochastic — the reference
DDIM	20–50 steps, deterministic, invertible
DPM-Solver++ / UniPC	10–20 steps, higher-order ODE solvers — the quality/speed default
Euler / Heun	flow-matching models' native solvers, few steps

```python
from diffusers import StableDiffusionXLPipeline

pipe = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0").to("cuda")
img = pipe("a lighthouse in fog", num_inference_steps=30,
           guidance_scale=7.0).images[0]
```

3.3 Flow matching: the modern formulation

Instead of noising to Gaussian and learning to denoise, learn a velocity field that carries samples of a simple distribution (noise) along straight *rectified* paths to data:

$$z_t = (1-t)\,z_{\text{noise}} + t\,x_{\text{data}}, \qquad \mathcal{L} = \|v_\theta(z_t, t) - (x_{\text{data}} - z_{\text{noise}})\|^2$$

The target velocity is constant along each path — easier to learn, fewer sampling steps, cleaner theory (it is a regression on straight lines). Sampling integrates the ODE $\frac{dz}{dt} = v_\theta(z_t, t)$ with Euler steps.

> **PITFALL**
> Three sampler notes: (1) steps and guidance interact — more steps want revisited guidance scales; benchmark the pair; (2) schedulers are checkpoints' metadata: `pipe.scheduler` is swappable

(`DPMSolverMultistep`, `EulerDiscrete`) and the `timestep_spacing` matters; (3) the `strength` parameter in img2img sets the starting timestep — it is the noise-vs-fidelity dial for edits.

CHAPTER 4

Conditioning: CLIP, Cross-Attention, Guidance

Unconditional diffusion draws *an* image; conditioning draws *your* image. Three mechanisms: CLIP's shared embedding space, cross-attention as the injection point, and classifier-free guidance as the amplifier.

4.1 CLIP: the shared space

CLIP trains an image encoder and a text encoder contrastively: matching image–text pairs pull together, mismatches push apart, over 400M pairs. The result is a joint space where similarity is measurable:

```
import torch, open_clip

model, _, prep = open_clip.create_model_and_transforms(
    "ViT-B-32", pretrained="laion2b_s34b_b79k")
tok = open_clip.get_tokenizer("ViT-B-32")
img = model.encode_image(prep(img_pil).unsqueeze(0))
txt = model.encode_text(tok(["a lighthouse in fog", "a cat"]))
img, txt = img / img.norm(dim=-1, keepdim=True), txt / txt.norm(dim=-1, keepdim=True)
print((img @ txt.T).softmax(dim=-1))     # which caption matches
```

CLIP gives: the text encoder that conditions diffusion, the zero-shot classifier (argmax over caption prompts), and the CLIP-score evaluation metric (Ch. 6's caveats apply).

4.2 Cross-attention: where text enters the image

The diffusion UNet injects text at every block via cross-attention: queries come from the image features, keys/values from the *text encoder's token embeddings* (not one pooled vector — every word gets its own channel):

$$\mathrm{CA}(Q_{\text{img}}, K_{\text{txt}}, V_{\text{txt}}) = \mathrm{softmax}\left(\frac{Q_{\text{img}} K_{\text{txt}}^{\top}}{\sqrt{d}}\right) V_{\text{txt}}$$

This per-token channel is why prompt structure matters: each token's embedding steers the image regions it attends to — prompt order, weights, and breaks have real geometric effects.

4.3 Classifier-free guidance (CFG)

Train the UNet both conditionally and unconditionally (drop the text 10% of the time), then extrapolate at sampling:

$$\epsilon = \epsilon_\theta(x_t, \varnothing) + w\,(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing))$$

with guidance weight w: $w = 1$ is the plain conditional model; $w = 3$–8 sharpens prompt alignment at the cost of diversity (the distribution narrows); $w = 15$ saturates into over-contrasted artifacts. CFG doubles compute (two forward passes) unless the conditional and unconditional branches are batched together.

PITFALL

Three conditioning traps: (1) CFG is trained at $w \approx$ its deployment scale — a checkpoint guided at $w = 7$ looks wrong at $w = 3$ and $w = 15$; read the model card; (2) prompt token limits (77 for CLIP-based SD) silently truncate long prompts — chunk or use T5-based models (SDXL/Flux) for long prompts; (3) negative prompts are the unconditional branch's text — "lowres, extra fingers" is a real engineering input, not a magic spell; it works via the same extrapolation arithmetic.

CHAPTER 5

Latent Diffusion and Fine-Tuning

Stable Diffusion's insight: run diffusion not on pixels but on a VAE's *latent* space — 8x smaller per side, 64x fewer values, on images that decode back convincingly. This chapter: the architecture, and the LoRA fine-tune recipe.

5.1 The architecture

Component	Job
VAE encoder	$512 \times 512 \times 3$ image \to $64 \times 64 \times 4$ latent
UNet (the diffusion model)	denoises latents, text-conditioned via cross-attention (Ch. 4)
VAE decoder	latents \to pixels
text encoder	CLIP/T5 embeddings for conditioning

Training runs the DDPM objective (Ch. 2) on VAE latents of pre-encoded images — the VAE is frozen; only the UNet (and the text encoder, optionally) learns.

5.2 LoRA fine-tuning

```
from diffusers import StableDiffusionXLPipeline
from peft import LoraConfig
import torch

pipe = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.bfloat16).to("cuda")
pipe.unet = _add_lora(pipe.unet)     # LoraConfig on attention layers
# train with diffusers' train_text_to_image_lora_sdxl.py -
# 100-1000 images of your concept/style, lr 1e-4, 1000-3000 steps
pipe.load_lora_weights("out/lora")
img = pipe("a lighthouse in the style of <srs>", num_inference_steps=30,
           cross_attention_kwargs={"scale": 1.0}).images[0]
```

LoRA targets attention layers only (the same `all-linear` instinct of the HF handbook, Ch. 7) — 50–200 MB adapters. Training data: 20–100 images for a style or subject; the concept token (<srs>) or just the style name binds the data.

> **PITFALL**
> Three fine-tuning traps: (1) overfitting a small set collapses CFG — diversity dies and every prompt yields the same composition; watch the guidance scale and step count; (2) training the VAE — leave it frozen (it is already excellent, and it drifts badly); (3) dataloader resolution mismatches — training at 512 and serving at 1024 with an SDXL checkpoint shifts the latent statistics; train and serve at the same resolution.

CONTROLNET, IN ONE PARAGRAPH

The biggest fine-tune-adjacent tool: a side network that adds a *spatial* condition (pose skeleton, edge map, depth) to the UNet via the same cross-attention hooks, trained on pairs (image, condition). It ships as a drop-in `controlnet=` argument in `StableDiffusionXLControlNetPipeline`. When prompts cannot express the composition you want, ControlNet is the answer.

CHAPTER 6

Vision-Language Models

VLMs give an LLM eyes. The recipe that won: a strong vision encoder + a small projector + the LLM — trained in two stages. This chapter is the architecture and the evaluation.

6.1 The architecture (LLaVA-style)

Component	Role
vision encoder (CLIP ViT)	image \to patch embeddings — frozen in stage 1
projector (MLP)	embeddings \to the LLM's token space — the trained bridge
LLM	the decoder; treats image tokens as ordinary input tokens

1. **stage 1 — alignment**: freeze encoder and LLM; train the projector on caption pairs (the MLP learns to translate).
2. **stage 2 — instruction tuning**: unfreeze the LLM (LoRA or full); train on visual instruction data (VQA, OCR, charts, documents).
3. **resolution**: native-resolution ViTs (variable aspect, tiles) replaced the fixed-336px compromise — document images need it.

```
from transformers import LlavaNextProcessor, LlavaNextForConditionalGeneration
import torch

proc = LlavaNextProcessor.from_pretrained(
    "llava-hf/llama3-llava-next-8b-hf")
model = LlavaNextForConditionalGeneration.from_pretrained(
    "llava-hf/llama3-llava-next-8b-hf",
    torch_dtype=torch.bfloat16, device_map="auto")

conversation = [{"role": "user", "content": [
    {"type": "image"},
    {"type": "text", "text": "What does the chart show?"}]}]
prompt = proc.apply_chat_template(conversation, add_generation_prompt=True)
inputs = proc(images=[chart_pil], text=prompt,
              return_tensors="pt").to(model.device)
out = model.generate(**inputs, max_new_tokens=200)
print(proc.decode(out[0][inputs["input_ids"].shape[1]:],
                  skip_special_tokens=True))
```

6.2 Evaluation

Axis	Instrument
general VQA	`MMMU`, `MMMU-Pro` (college-level, multimodal)
document/OCR	`DocVQA`, `ChartQA` — the production-critical slice
hallucination	`POPE` (does the object exist?), object-hallucination rates
counting/position	known weak spots — test explicitly before shipping

PITFALL

Three VLM traps: (1) resolution starvation — a 1MP photo downsampled to 336px loses the text your task needs; native-resolution models or tiling is the fix; (2) hallucination under uncertainty — VLMs name plausible objects that are not there; POPE-style existence checks catch it; (3) multi-image and ordering — most open VLMs are single-image first-class; multi-image reasoning varies wildly — benchmark your case.

THE ONE-SENTENCE SUMMARY

A VLM is your fine-tuned LLM with a CLIP encoder grafted through a projector — stage-1 aligns the graft, stage-2 teaches the task, and evaluation must include the hallucination and resolution axes, because those are the ones that fail in production.

APPENDIX A

Cheat Sheet

A.1 Diffusion

Piece	Formula
forward (any step)	$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon$
training	MSE between added and predicted noise
DDIM step	deterministic; $x_{t'}$ from predicted x_0
flow matching	$z_t = (1 - t) z_{\text{noise}} + tx$; regress the velocity
CFG	$\epsilon = \epsilon_\varnothing + w(\epsilon_c - \epsilon_\varnothing)$; $w = 3$–8

A.2 Latent diffusion

Component	Note
VAE	frozen; 8x spatial compression; do not fine-tune casually
UNet + cross-attention	text enters per-token (CLIP/T5 embeddings)
LoRA	attention layers only; 50–200 MB; 100–1000 images
ControlNet	spatial conditions (pose, depth, edges) via the same hooks

A.3 VLMs

Stage	What trains
1: alignment	projector only, on captions; encoder + LLM frozen
2: instruction	LLM (LoRA or full) + projector, on VQA/instruction data

Eval axis	Suite
general	MMMU, MMMU-Pro
documents	DocVQA, ChartQA
hallucination	POPE-style existence checks
resolution	native-res or tiled encoders for documents

A.4 Idioms worth memorizing

```
x_t = abar[t].sqrt() * x0 + (1 - abar[t]).sqrt() * eps   # forward
eps_pred = unet(x_t, t)                                  # the whole model
x0 = (x_t - (1 - abar[t]).sqrt() * eps_pred) / abar[t].sqrt()
img @ txt.T                                              # CLIP similarity
pipe(..., guidance_scale=7.0, negative_prompt="...")     # CFG in practice
```

THE HABITS THAT MATTER

Read which parameterization a checkpoint uses before touching it (noise / x_0 / velocity). Train and serve at the same resolution. Evaluate on your domain's VQA and on object-existence probes. Fine-tune adapters, freeze the VAE. And when prompts cannot express the composition, reach for ControlNet — not for a bigger model.